HỌC PHẦN INT1306 · THỰC HÀNH 09/10/2026

Thực hành Cấu trúc dữ liệu và giải thuật: Điều phối giao đơn bằng Julia

Một bài toán, một chương trình Julia, bốn hàm trọng tâm.

Sinh viên hoàn thiện một chương trình Julia điều phối giao đơn trong khuôn viên trường. Chương trình đọc hai tệp CSV, chuẩn hóa danh mục đơn, tra cứu theo mã, tính đường ngắn nhất từ bản đồ, chọn các đơn trong ngân sách thời gian, quản lý giao và hoàn tác, sau đó xuất kết quả. Bốn hàm cần hoàn thiện tham gia cùng một luồng dữ liệu; các cấu trúc danh sách, hàng đợi, ngăn xếp, BST, DFS và menu đã được cung cấp để sinh viên tập trung vào ý tưởng và tính đúng đắn của thuật toán.

CSV Đơn và bản đồ	**Danh mục** Sắp xếp, tra cứu	**BFS** Đường → thời gian
DP Chọn trong B	**Giao / hoàn tác** Queue / stack	**CSV kết quả** Cùng trạng thái

> Khung thời lượng: 180 phút. Giảng viên có hướng dẫn rút gọn 90 phút và mở rộng 225 phút trong tài liệu riêng.

Mục tiêu và tổ chức buổi học

- Nêu được đầu vào, đầu ra và tiền điều kiện của bốn hàm cần hoàn thiện; phân biệt dữ liệu đơn hàng, chi phí tính từ bản đồ và trạng thái giao hàng.
- Giải thích được cách chia để trị của sắp xếp trộn và điều kiện mảng có thứ tự của tìm kiếm nhị phân; truy vết các bước trên cùng danh mục ba đơn.
- Hiện thực được sắp xếp trộn trả một dãy mới và tìm kiếm nhị phân trả đúng bản ghi hoặc nothing; kiểm tra trường hợp rỗng, một phần tử và mã không tồn tại.
- Hiện thực được BFS trên đồ thị không trọng số, lưu khoảng cách và nút cha, dựng lại đường đi; tính đúng thời gian đi–về và phục vụ của từng đơn.
- Thiết lập được trạng thái, công thức chuyển và truy vết của quy hoạch động ba lô 0/1; đối chiếu giá trị tối ưu với vét cạn trên dữ liệu nhỏ.
- Phân tích được sự thay đổi của FIFO, ngăn xếp, BST và các tổng khi giao hoặc hoàn tác; kiểm tra bất biến không mất, không lặp đơn trong một lượt chạy.
- Đánh giá được kết quả thực nghiệm, độ phức tạp và các lỗi thường gặp; trình bày một báo cáo ngắn có lệnh chạy, bằng chứng kiểm thử và phiên bản mã nguồn trên GitHub.

Phút	Trọng tâm	Hoạt động	Kết quả
10	Xác định bài toán và đọc sơ đồ xử lý	Giảng viên giới thiệu một lượt giao đơn; sinh viên chỉ ra nơi các thuật toán truyền dữ liệu cho nhau và ghi đầu vào, đầu ra.	Một sơ đồ CSV → danh mục → BFS → chi phí → chọn đơn → giao/hoàn tác → CSV.
15	Tạo và đọc hai CSV	Chạy chế độ --data, mở bộ MINI001, đọc tiêu đề và ba dòng đơn; kiểm tra bản đồ vô hướng và miền dữ liệu.	Hai CSV được tạo; sinh viên giải thích đúng bốn cột đơn hàng và hai cột cạnh.
25	Nhiệm vụ 1: Sắp xếp trộn	Truy vết danh mục [3,1,2], hoàn thiện merge_sort và chạy kiểm thử --phase=sort; ghi điều kiện dừng và cách trộn.	Dãy ID tăng [1,2,3]; dãy đầu vào không đổi; kiểm thử pha sort và nhận xét lỗi đã sửa.
20	Nhiệm vụ 2: Tìm kiếm nhị phân	Hoàn thiện binary_search trên danh mục ID tăng; thử ID 2, ID 99, mảng rỗng và chạy --phase=search.	ID 2 trả đúng bản ghi; ID 99 và mảng rỗng trả nothing; một bảng left,mid,right.

Phút	Trọng tâm	Hoạt động	Kết quả
30	Nhiệm vụ 3: BFS và thời gian giao	Quan sát hoạt hình FIFO, tự tính khoảng cách, hoàn thiện bfs_routes và chạy --phase=bfs; tái dùng merge_sort để sắp ưu tiên theo chi phí vừa tính.	Khoảng cách từ 1 là [0,1,1,2,2,3]; thời gian ba đơn là [3,5,7]; các đường đi hợp lệ.
35	Nhiệm vụ 4: Quy hoạch động chọn đơn	Điền một số ô F(i,b), hoàn thiện knapsack và truy vết; chạy --phase=dp, đối chiếu bộ vét cạn đã cho.	Ngân sách 8 chọn ID 1 và 2, thời gian 8, lợi ích 14; giải thích quyết định không chọn ID 3.
20	Chạy ứng dụng trọn luồng	Chạy --mini và --phase=app; xem kế hoạch, giao một đơn, hoàn tác, giao lại, xuất CSV; theo dõi cùng một ID qua tất cả các bước.	Trạng thái sau giao ID 1 là thời gian 3, lợi ích 6; sau hoàn tác trở về 0,0 và hàng đợi [1,2].
10	Kiểm tra kiến thức cuối buổi	Trả lời độc lập năm câu hỏi ngắn; giảng viên dùng câu trả lời để kiểm tra tiền điều kiện và khả năng giải thích thuật toán.	Phiếu trả lời năm câu, có lý giải bằng dữ liệu của bài.
15	Ghi kết quả, lưu mã và chuẩn bị báo cáo	Chạy kiểm thử toàn bộ, ghi kết quả thực tế, lưu mọi mã trên GitHub và lập dàn ý báo cáo sáu phần. Sau buổi học, hoàn thiện báo cáo và notebook Julia mẫu để nộp và trình bày.	URL GitHub, commit, mã Julia, kết quả kiểm thử và dàn ý báo cáo; kế hoạch hoàn thiện notebook sau buổi.

Một bài toán dùng chung dữ liệu

Ba đơn A, B, C dùng chung bản đồ. Mỗi chuyến đi và về kho 1; ngân sách B=8.

ID	Điểm giao	Phục vụ	Lợi ích
1	2	1	6
2	4	1	8
3	6	1	10

Đọc đơn → tra cứu và tạo thứ tự → BFS tính thời gian → DP chọn đơn → đưa kế hoạch vào hàng đợi → giao/hoàn tác → xuất CSV.

Nếu đường thay đổi, thời gian có thể thay đổi; kế hoạch và hàng đợi sử dụng thời gian mới. Bài tập yêu cầu bảo toàn mối liên hệ này.

Chuẩn bị trước khi cài thuật toán

- Sinh viên đã biết biến, kiểu dữ liệu, mảng, vòng lặp, điều kiện, hàm và gọi hàm trong Julia. Các mảng Julia trong bài dùng chỉ số bắt đầu từ 1.
- Chuẩn bị Julia chạy được trên máy và một thư mục chứa TH_2026_10_09_Sinh_vien.jl cùng TH_2026_10_09_Kiem_thu.jl. Mở cửa sổ lệnh tại thư mục này. Các thao tác thực hành dùng Julia Base; không phải cài thư viện CSV hoặc phần cứng.
- Chạy julia TH_2026_10_09_Sinh_vien.jl --data để tạo và đọc bộ mẫu MINI001 trước khi làm thuật toán. Chạy julia TH_2026_10_09_Kiem_thu.jl --phase=data để kiểm tra bước dữ liệu.
- Chỉ sửa bốn vị trí TODO trong merge_sort, binary_search, bfs_routes và knapsack. Đọc các kiểu Order và Session cùng hàm hỗ trợ đã cho; không thay định dạng CSV, chữ ký hàm hoặc kết quả trả về.
- Không gọi chế độ tích hợp --mini hoặc --demo để kết luận bài đã hoàn thành khi bốn hàm còn thiếu. Mỗi pha có kiểm thử riêng; lời báo CHUA_HOAN_THANH cho biết còn vị trí cần hiện thực.
- Bộ ba đơn MINI001 dùng để học và tính tay trong buổi. Dữ liệu 8–10 đơn theo MSSV dành cho bước phát triển sau khi bộ mẫu chạy đúng. Phần trình bày bằng Julia là bắt buộc: sau buổi học, hoàn thiện notebook Julia/Pluto mẫu gọi lại cùng lõi; nếu máy chưa cài Pluto, có thể chạy trực tiếp tệp notebook .jl bằng Julia. Không viết một phiên bản thuật toán khác.

Khởi động chương trình bằng Julia

Giải nén bộ sinh viên; mở Julia tại thư mục chứa các file. Khung chạy được phần dữ liệu ngay. Những lời gọi cần thuật toán chưa hoàn thiện sẽ báo CHUA_HOAN_THANH.

```
julia TH_2026_10_09_Sinh_vien.jl --data
julia TH_2026_10_09_Kiem_thu.jl --phase=data
```

Trong cửa sổ Julia, nạp khung và xem dữ liệu bằng các lệnh sau. Đây là phần đã được cung cấp, không cần cài gói CSV ngoài.

```
# Trong Julia REPL; chạy một lần để nạp khung.
include("TH_2026_10_09_Sinh_vien.jl")
using .GiaoDon
folder = mini_data()
data = read_data(folder)
p = mini_profile()
```

```
# Tiền điều kiện: đã chạy setup. Hai CSV này chỉ chứa số nguyên.
orders = data.orders
edges = data.edges
println([(o.id,o.destination,o.service,o.benefit) for o in orders])
println(edges)
```

Hai CSV quy định chỉ có số nguyên và tiêu đề cố định. Hàm đọc kiểm tra định dạng và giá trị; không áp dụng cách tách bằng dấu phẩy cho CSV tổng quát có ô chứa dấu phẩy hoặc xuống dòng.

Sau khi lưu phần code vừa sửa, khởi động lại Julia REPL và chạy lại các lệnh nạp khung, đọc dữ liệu ở trên trước khi thử thuật toán. Việc lưu file không tự cập nhật các hàm trong phiên Julia đang mở. Các lệnh julia ... --phase=... trong terminal mở một phiên mới cho mỗi lần kiểm tra.

Cách chạy tệp và include: Julia Manual — Getting Started, https://docs.julialang.org/en/v1/manual/getting-started/. Phép đọc/ghi: Julia Base — I/O and Network, https://docs.julialang.org/en/v1/base/io-network/.

Minh họa động từ cùng bộ dữ liệu

Chọn một thuật toán, quan sát từng bước và ghi trạng thái trước khi viết Julia. Khung này minh họa thuật toán; chương trình thực hành chạy bằng Julia.

CÙNG MỘT ỨNG DỤNG GIAO ĐƠN

Chọn đơn bằng quy hoạch động

Giới hạn 8 đơn vị thời gian; mỗi đơn chỉ được chọn một lần

1. Sắp xếp đơn | 2. Tính thời gian | **3. Chọn kế hoạch** | 4. Giao và hoàn tác

> Kế hoạch tối ưu chọn A và B: thời gian 3 + 5 = 8, lợi ích 6 + 8 = 14. Mỗi đơn được lấy tối đa một lần.

F[i,b] = max(F[i−1,b], lợi_ích[i] + F[i−1,b−chi_phí[i]]) khi đủ thời gian; nếu không đủ, giữ F[i−1,b].

Bảng F — cột là giới hạn thời gian b từ 0 đến 8

i — đơn vừa xét	0	1	2	3	4	5	6	7	8
0 — chưa xét đơn	0	0	0	0	0	0	0	0	0
1 — A (t=3, v=6)	0	0	0	6	6	6	6	6	6
2 — B (t=5, v=8)	0	0	0	6	6	8	8	8	14
3 — C (t=7, v=10)	0	0	0	6	6	8	8	10	• 14

● ô đang xét; ◇ ô nguồn ở hàng trước; · chưa tính. Chỉ số trong bảng là ký hiệu toán học từ 0; khi cài đặt Julia, dùng vị trí mảng từ 1.

Đơn đã truy vết: **[1, 2]** Giá trị F[3,8]: **14**

Lấy giá trị từ hàng i−1 bảo đảm đơn i không được chọn lại. Bảng cần O(nB) phép tính; BFS quyết định chi phí nên thay bản đồ có thể thay kế hoạch.

Câu hỏi quan sát: Vì sao công thức lấy F của hàng trước, không lấy lại chính hàng đang tính?

1. Bài toán, mô hình dữ liệu và CSV

Lý thuyết sử dụng

- Đặc tả bài toán xác định dữ liệu hợp lệ và kết quả cần đạt. Bản ghi Order lưu mã đơn id, điểm giao, thời gian phục vụ và lợi ích. Bản đồ là đồ thị vô hướng không trọng số; mỗi cạnh cần một đơn vị thời gian. Chi phí của đơn phải được tính từ bản đồ, không gán sẵn trong CSV.
- Mỗi chuyến chỉ giao một đơn: xuất phát tại điểm 1, đi tới điểm giao rồi trở về điểm 1. Nếu BFS cho số cạnh ít nhất h_i, thời gian thực hiện là τ_i = 2h_i + s_i, với s_i là thời gian phục vụ. Ngân sách B và τ_i đều là số nguyên, cùng một đơn vị thời gian giả định.
- don_hang.csv có tiêu đề id,diem_giao,thoi_gian_phuc_vu,loi_ich. ban_do.csv có tiêu đề u,v; mỗi cạnh chỉ ghi một lần. Khi dựng danh sách kề, chương trình thêm cả u vào danh sách của v và v vào danh sách của u.
- Tạo CSV đơn giản bằng open(...,"w") do io, println tiêu đề rồi println từng dòng các số nguyên cách nhau bởi dấu phẩy. Đọc bằng readlines hoặc eachline, kiểm tra tiêu đề, dùng split(line, ',') và parse(Int, ...) cho từng trường. Cách này dành cho CSV số nguyên của bài; không áp dụng nguyên trạng cho văn bản có dấu phẩy nằm trong dấu ngoặc kép.
- Bản đồ mẫu có các cạnh (1,2),(1,3),(2,4),(3,4),(3,5),(4,6),(5,6). Các cấu trúc đã cung cấp giữ cùng một tập mã đơn: danh mục và BST phục vụ tra cứu, danh sách liên kết giữ kế hoạch, FIFO giữ các đơn chờ, ngăn xếp giữ lịch sử giao.
- Sinh viên hoàn thiện bốn thuật toán. Mã hỗ trợ sinh/đọc CSV, DFS, cấu trúc dữ liệu, đệ quy tính tổng, vét cạn và menu được cung cấp để nối thành một ứng dụng ngay trong buổi; sinh viên vẫn cần đọc và giải thích vai trò của chúng.

Ví dụ và suy luận

- Bộ MINI001 có ba dòng: 1,2,1,6; 2,4,1,8; 3,6,1,10. Các ID 1,2,3 tương ứng đơn A,B,C. Đây là dữ liệu kiểm thử thu nhỏ, không phải dữ liệu cá nhân theo MSSV.
- Với đơn A, đường đi 1→2 có một cạnh; thời gian cả đi, về và phục vụ là 2×1+1=3. Đơn B tới điểm 4 có hai cạnh nên τ=5. Đơn C tới điểm 6 có ba cạnh nên τ=7. Ngân sách mẫu B=8.
- Sau khi chạy chế độ --data, mở hai CSV và đối chiếu trực tiếp các dòng. Chưa chỉnh τ trong CSV vì chương trình phải tính τ từ kết quả BFS ở nhiệm vụ 3.

Lỗi cần tránh

- Đọc dòng tiêu đề như dữ liệu số hoặc bỏ qua dòng có lỗi mà không báo vị trí.
- Chỉ thêm một chiều của cạnh, làm sai ý nghĩa đồ thị vô hướng.
- Nhầm thời gian một lượt đi với thời gian cả đi và về; thiếu hệ số 2 trong τ_i.
- Tạo các bộ đơn độc lập cho từng thuật toán, khiến kết quả sắp xếp, đồ thị và hàng đợi không còn liên quan.

- Sửa dữ liệu cá nhân để tạo một kết quả mong muốn thay vì kiểm tra thuật toán trên dữ liệu đã sinh.

Chi phí và điều kiện áp dụng

- Đọc n bản ghi số nguyên có độ dài hữu hạn cần O(n) theo số bản ghi; tổng quát, chi phí đọc phụ thuộc số ký tự của tệp.
- Danh sách kề dùng O(V+E) bộ nhớ. Bản đồ vô hướng lưu mỗi cạnh trong hai danh sách kề.
- Danh sách liên kết, hàng đợi và ngăn xếp giữ các ID kế hoạch, không nhân bản toàn bộ bản ghi và tuyến đường.

2. Sắp xếp trộn: Chuẩn hóa danh mục và tái dùng để xếp ưu tiên

Lý thuyết sử dụng

- Sắp xếp trộn chia dãy thành hai phần nhỏ hơn, sắp từng phần bằng đệ quy rồi trộn hai dãy đã có thứ tự. Điều kiện dừng là dãy có không quá một phần tử. Đây là phương pháp chia để trị.
- Hàm merge_sort(a, before) nhận dãy a và hàm so sánh before; trả dãy mới chứa đúng các phần tử ban đầu theo thứ tự đã yêu cầu, không làm thay đổi a. So sánh phải nhất quán. Khi hai phần tử ngang nhau, lấy phần tử ở nhánh trái trước để giữ tính ổn định.
- Trong bước đầu, sắp danh mục đơn theo ID tăng để tạo tiền điều kiện cho tìm kiếm nhị phân. Cùng hàm sắp xếp được dùng để chuẩn hóa thứ tự láng giềng trong danh sách kề, giúp lượt truy vết BFS dễ tái lập.
- Sau nhiệm vụ BFS, tái dùng merge_sort để sắp ưu tiên giảm theo lợi ích/τ. So sánh hai đơn a,b bằng lợi_ích_a×τ_b và lợi_ích_b×τ_a; khi bằng nhau dùng quy tắc ID trong hồ sơ. Với bộ mini các tỷ số khác nhau nên thứ tự ưu tiên không phụ thuộc quy tắc hòa.
- Bất biến của bước trộn: phần đã xuất có thứ tự, chứa đúng các phần tử đã lấy từ hai nhánh; phần tử tiếp theo phải là phần tử nhỏ hơn theo before trong hai đầu chưa lấy. Sau khi một nhánh hết, chép toàn bộ phần còn lại của nhánh kia.

Ví dụ và suy luận

- Danh mục thử có ID [3,1,2]. Chia thành [3] và [1,2]; nhánh phải chia [1] và [2] rồi trộn thành [1,2]. Trộn [3] với [1,2]: lấy 1, lấy 2, chép 3; kết quả [1,2,3].
- Sau khi có τ=[3,5,7], ba tỷ số ưu tiên là 6/3=2; 8/5=1,6; 10/7≈1,43. Thứ tự giao ưu tiên là [1,2,3]; tập được chọn sẽ do quy hoạch động quyết định ở nhiệm vụ 4.
- Tính tay một phép so sánh: 6×5=30 lớn hơn 8×3=24 nên đơn 1 đứng trước đơn 2. Không cần chuyển thành số thực để so sánh.

Lỗi cần tránh

- Không giảm kích thước nhánh đệ quy, khiến hàm không dừng.
- Bỏ phần còn lại sau khi một nhánh đã hết, làm mất đơn.
- Dùng sort hoặc sort! để thay nhiệm vụ sắp xếp đang được đánh giá.
- Sắp theo ưu tiên rồi dùng chính dãy đó để tìm kiếm nhị phân theo ID; hai khóa sắp xếp khác nhau nên tiền điều kiện bị phá.
- Trộn vào dãy đầu vào và vô tình thay danh mục dùng chung; không kiểm tra tính ổn định khi comparator hòa.

Chi phí và điều kiện áp dụng

- Thời gian O(n log n): có khoảng $\log_2 n$ tầng chia và O(n) thao tác trộn trên mỗi tầng.
- Một hiện thực dùng bộ đệm trộn chung cần O(n) bộ nhớ phụ và O(log n) độ sâu ngăn xếp. Nếu dùng nhiều lát cắt và vector tạm, tổng lượng cấp phát có thể lớn hơn; cần mô tả đúng hiện thực của mình.
- Tự đếm số phép so sánh trên ba đơn để hiểu hoạt động; số đo một ví dụ nhỏ không thay cho phân tích khi n tăng.

NV1 · Hoàn thiện merge_sort(a, before)

- Đầu vào: một dãy và hàm before. Đầu ra: vector mới cùng các phần tử, có thứ tự theo before; giữ dãy đầu vào không đổi.
- Xử lý dãy rỗng và một phần tử; đệ quy hoặc chia nhánh đúng, trộn đủ phần tử, ổn định khi các khóa bằng nhau.
- Trên cùng ba bản ghi Order, thử thứ tự ban đầu [3,1,2] và comparator ID tăng. Không xây dựng một bộ dữ liệu ứng dụng khác.
- Ghi ngắn ý tưởng chia, điều kiện dừng và bất biến bước trộn trước khi chạy kiểm thử.

Gợi ý triển khai

- Nếu length(a)≤1, trả bản sao có cùng kiểu phần tử.
- Giữ hai chỉ số chỉ phần tử chưa lấy của hai nhánh. Chỉ đọc phần tử khi chỉ số còn trong phạm vi.
- Nếu phần tử nhánh phải đứng trước nhánh trái thì lấy bên phải; còn lại lấy bên trái, kể cả trường hợp comparator hòa.
- Sau vòng so sánh, chép phần dư của cả hai nhánh.

```
function merge_sort(a::AbstractVector, before::Function)
    # Sinh viên cài đặt tại vị trí TODO trong file chung.
end
```

Lời gọi kiểm tra trong cùng chương trình

```
# Tiền điều kiện: đã chạy setup và hoàn thiện TODO 1.
a = merge_sort([3,1,2], (<))
@assert a == [1,2,3]
by_id = merge_sort(data.orders[[3,1,2]],(x,y)->x.id < y.id)
```

```
julia TH_2026_10_09_Kiem_thu.jl --phase=sort
```

- Chạy julia TH_2026_10_09_Kiem_thu.jl --phase=sort. Ghi số kiểm tra thực sự đạt và lỗi còn lại.

- ID [3,1,2] phải thành [1,2,3]; các bản ghi vẫn giữ đúng điểm giao, phục vụ và lợi ích.
- Dãy rỗng và một phần tử không gây lỗi; dãy đầu vào giữ nguyên; phần tử không bị thiếu hoặc lặp.
- Một comparator coi hai khóa bằng nhau không được làm đảo thứ tự tương đối của chúng.

3. Tìm kiếm nhị phân: Tra đúng đơn trong cùng danh mục

Lý thuyết sử dụng

- Tìm kiếm nhị phân thu hẹp một khoảng ứng viên trên dãy đã sắp theo khóa tìm kiếm. Trong bài, dãy phải tăng theo Order.id, mã đơn phải duy nhất. Mỗi bước so sánh với phần tử giữa rồi loại một nửa không thể chứa mã cần tìm.
- Hàm binary_search(a::Vector{Order}, id::Int) trả bản ghi Order tìm được hoặc nothing nếu không tồn tại. Hàm chỉ tra cứu, không cập nhật kế hoạch hoặc trạng thái giao.
- Đặt left=1, right=length(a). Khi left≤right, lấy mid=left+(right−left)÷2. Nếu khóa bằng id thì trả bản ghi; nếu khóa nhỏ hơn id thì left=mid+1; ngược lại right=mid−1. Khoảng còn xét giảm sau mỗi lần chưa tìm thấy.
- Bất biến: nếu id tồn tại trong danh mục thì vị trí của nó nằm trong đoạn còn xét. Sau khi left>right, đoạn rỗng nên trả nothing. Với mảng rỗng, right=0 nên không đọc a[1].
- BST đã cung cấp cũng tra cứu cùng id trong cùng danh mục. Hai cách phải trả cùng thông tin; đây là phép đối chiếu của ứng dụng, không phải hai danh mục độc lập. BST có chi phí O(h) theo chiều cao h, không mặc nhiên luôn O(log n).

Ví dụ và suy luận

- Trên dãy ID [1,2,3], tìm ID 2: left=1, right=3, mid=2; trả bản ghi (id=2, điểm giao=4, phục vụ=1, lợi ích=8).
- Tìm ID 3: tại mid=2 thấy 2<3 nên left=3; lần sau mid=3 và tìm thấy. Tìm ID 99: khoảng lần lượt thu hẹp rồi rỗng; trả nothing.
- Ghi một bảng gồm các cột lần lặp, left, mid, right, khóa giữa, quyết định. Dùng dữ liệu ID của cùng danh mục đọc từ CSV.

Lỗi cần tránh

- Gọi tìm kiếm nhị phân trên dãy đã sắp theo lợi ích/τ thay vì theo ID.
- Cập nhật left=mid hoặc right=mid khi cần loại mid, khiến khoảng không giảm.
- Trả chỉ số hoặc false khi hợp đồng yêu cầu Order hoặc nothing.
- Đọc a[mid] trước khi kiểm tra left≤right, gây lỗi với mảng rỗng.
- Cho rằng BST luôn cân bằng; chèn dãy ID tăng vào BST cơ bản có thể tạo cây lệch.

Chi phí và điều kiện áp dụng

- Tìm kiếm nhị phân cần O(log n) thời gian và O(1) bộ nhớ phụ với hiện thực vòng lặp.
- Chi phí sắp danh mục ban đầu là O(n log n); chi phí này không phát sinh lại sau mỗi lần tra cứu nếu dữ liệu không đổi.

- Tra cứu BST cơ bản cần O(h), tốt khi h nhỏ và xấu nhất O(n). Chi phí giao còn gồm tra cứu này và các cập nhật trạng thái.

NV2 · Hoàn thiện binary_search(a::Vector{Order}, id::Int)

- Đầu vào: danh mục Order có ID tăng và mã cần tìm. Đầu ra: Order tương ứng hoặc nothing. Không thay nội dung dãy.
- Hiện thực bằng hai biên và điểm giữa; mỗi lần chưa khớp phải giảm đoạn ứng viên.
- Thử ID đầu, ID giữa, ID cuối, mã không có và mảng rỗng; đối chiếu thông tin tìm được với BST đã cho.
- Giải thích bằng một câu vì sao dãy ưu tiên dùng khi giao không thể tự động thay danh mục ID tăng.

Gợi ý triển khai

- Chỉ truy cập a[mid] trong vòng lặp left≤right.
- Tính mid bằng left+(right−left)÷2.
- So sánh a[mid].id với id, không so sánh toàn bộ bản ghi.
- Khi đoạn đã rỗng, trả nothing; không sửa danh mục để biểu diễn việc không tìm thấy.

```
function binary_search(a::Vector{Order},id::Int)
    # Sinh viên cài đặt tại vị trí TODO trong file chung.
end
```

Lời gọi kiểm tra trong cùng chương trình

```
# Tiền điều kiện: đã chạy sort_call và hoàn thiện TODO 2.
found = binary_search(by_id,2)
@assert found !== nothing && found.id == 2
@assert binary_search(by_id,99) === nothing
```

```
julia TH_2026_10_09_Kiem_thu.jl --phase=search
```

- Chạy julia TH_2026_10_09_Kiem_thu.jl --phase=search.
- Trên bộ mini, tìm ID 2 phải trả điểm giao 4, phục vụ 1 và lợi ích 8.
- Tìm ID 99 hoặc tìm trên mảng rỗng phải trả nothing.
- Thông tin BST và binary_search cho từng ID phải khớp; mọi trạng thái giao giữ nguyên sau tra cứu.

4. BFS: Từ bản đồ tới thời gian thực hiện đơn

Lý thuyết sử dụng

- Đồ thị biểu diễn các điểm và đường nối. DFS đã cho kiểm tra khả năng tới được; BFS tìm số cạnh ít nhất từ nguồn trong đồ thị không trọng số. Khi các cạnh có trọng số khác nhau, không được dùng kết luận này nguyên trạng.
- BFS dùng hàng đợi FIFO: đỉnh phát hiện trước được xử lý trước. Khởi tạo khoảng cách mọi đỉnh bằng −1, khoảng cách nguồn bằng 0. Khi phát hiện láng giềng chưa tới, đặt distance[v]=distance[u]+1, lưu parent[v]=u và đưa v vào hàng đợi.
- Đánh dấu khoảng cách ngay khi đưa đỉnh vào hàng đợi để mỗi đỉnh chỉ vào một lần. Bất biến: hàng đợi xử lý các đỉnh theo lớp khoảng cách không giảm; lần đầu tới một đỉnh là bằng số cạnh ít nhất.
- bfs_routes(graph, source=1) trả named tuple có ba trường distance, parent, routes. distance và parent là các vector số nguyên; routes[v] là vector các đỉnh trên một đường ngắn nhất từ source tới v. Đỉnh không tới được có distance[v]=−1; không biến giá trị đó thành thời gian giao hợp lệ.
- Dựng đường đi bằng cách lần theo parent từ đích về nguồn rồi đảo thứ tự. Đỉnh nguồn có đường đi chỉ chứa chính nó. Cần quy ước rõ nút cha của nguồn theo mã hỗ trợ và tránh vòng lặp truy vết không dừng.
- Với từng Order, lấy h=distance[điểm giao] rồi tính τ=2h+phục vụ. Mã khung lưu costs là từ điển Dict{Int,Int} ánh xạ ID→τ; đọc chi phí bằng costs[o.id], không dùng vị trí của đơn trong dãy làm khóa. Chính costs này được truyền vào knapsack; tuyến đường và chi phí còn dùng khi giao, hoàn tác và xuất kết quả.

Ví dụ và suy luận

- Với láng giềng tăng theo ID: bắt đầu FIFO [1], distance[1]=0; xử lý 1 phát hiện 2 và 3 với khoảng cách 1; xử lý 2 phát hiện 4 với khoảng cách 2; xử lý 3 phát hiện 5 với khoảng cách 2; xử lý 4 phát hiện 6 với khoảng cách 3.
- Khoảng cách tới các điểm 1..6 là [0,1,1,2,2,3]. Một bộ đường đi hợp lệ cho ba đơn là 1→2, 1→2→4, 1→2→4→6. Các đường cùng số cạnh như 1→3→4 cũng được chấp nhận.
- Từ cùng ba đơn CSV, tính costs=Dict(1=>3,2=>5,3=>7), tương ứng thời gian [3,5,7] theo ID 1,2,3. Tái dùng merge_sort để thu thứ tự ưu tiên [1,2,3] trước khi lập kế hoạch.
- Quan sát hoạt hình từng thao tác lấy đầu FIFO, phát hiện đỉnh và ghi parent. Dừng hình ở lúc phát hiện điểm 6 rồi tự giải thích vì sao khoảng cách bằng 3.

Lỗi cần tránh

- Đánh dấu khi lấy ra thay vì khi đưa vào hàng đợi, gây đưa một đỉnh nhiều lần.
- Dùng ngăn xếp để lấy đỉnh tiếp theo nhưng vẫn gọi kết quả là đường ngắn nhất BFS.
- Nhầm vị trí của đơn trong mảng với số hiệu điểm giao khi đọc distance.

- Dùng costs gán tay [3,5,7] trong hàm lập kế hoạch, khiến bản đồ thay đổi mà phương án không đổi.
- Cho rằng chỉ một tuyến đường cụ thể là đúng dù đồ thị có nhiều đường ngắn nhất.

Chi phí và điều kiện áp dụng

- BFS và DFS trên danh sách kề cần O(V+E) thời gian. BFS cốt lõi dùng O(V) bộ nhớ cho khoảng cách, nút cha và hàng đợi, ngoài bộ nhớ đồ thị.
- Dựng một tuyến đường tốn O(độ dài đường). Nếu hàm trả sẵn routes cho mọi đỉnh, bộ nhớ các đường đi có thể lên tới $O(V^2)$, và phải tính thêm tổng độ dài các đường được dựng.
- Tính costs cho n đơn từ khoảng cách đã có cần O(n); không cần chạy BFS lại riêng cho từng đơn nếu tất cả cùng xuất phát tại điểm 1.

NV3 · Hoàn thiện bfs_routes(graph, source=1)

- Đầu vào: danh sách kề hợp lệ và nguồn. Đầu ra: named tuple distance, parent, routes với khoảng cách số cạnh ít nhất và một đường tương ứng cho mỗi đỉnh tới được.
- Dùng FIFO, đánh dấu ngay khi phát hiện, lưu nút cha và truy vết đúng hướng nguồn→đích.
- Tính τ cho từng đơn từ distance và lưu trong Dict theo đúng ID. Chuyển chính costs này sang nhiệm vụ chọn đơn.
- Kiểm tra nguồn, đồ thị có chu trình và một đỉnh không tới được. Giải thích trường hợp nhiều tuyến đường tối ưu.

Gợi ý triển khai

- Khởi tạo distance bằng −1. Với đỉnh đang xét u, chỉ phát hiện v khi distance[v] còn −1.
- Khi phát hiện v, ghi parent và distance trước khi đưa vào FIFO.
- Để dựng routes[v], đi ngược các parent từ v về nguồn, lưu tạm rồi đảo dãy; đỉnh không tới được giữ đường rỗng.
- Sau nhiệm vụ 1, danh sách láng giềng đã có thứ tự; việc này giúp theo dõi một lượt chạy nhưng không thay tính đúng của khoảng cách BFS.

```
function bfs_routes(graph,source=1)
    # Sinh viên cài đặt tại vị trí TODO trong file chung.
end
```

Lời gọi kiểm tra trong cùng chương trình

```
# Tiền điều kiện: đã chạy setup và hoàn thiện TODO 1, TODO 3.
graph = adjacency(data.edges,0)
routes = bfs_routes(graph,1)
@assert routes.distance == [0,1,1,2,2,3]
costs = Dict(o.id=>2*routes.distance[o.destination]+o.service for o in data.orders)
@assert [costs[i] for i in 1:3] == [3,5,7]
```

```
julia TH_2026_10_09_Kiem_thu.jl --phase=bfs
```

- Chạy julia TH_2026_10_09_Kiem_thu.jl --phase=bfs.
- Đồ thị mini phải cho distance=[0,1,1,2,2,3] và [costs[id] for id in 1:3]==[3,5,7].
- Tuyến đường bắt đầu tại nguồn, kết thúc tại đích, từng cặp liên tiếp phải có cạnh; số cạnh bằng distance[đích].
- Đỉnh không tới được có distance=−1 và đường rỗng; chương trình từ chối lập kế hoạch giao tới điểm đó.
- Không so sánh cứng chuỗi đường mẫu khi có nhiều đường ngắn nhất; so sánh khoảng cách và tính hợp lệ.

5. Quy hoạch động: Chọn các đơn trong ngân sách thời gian

Lý thuyết sử dụng

- Bài toán ba lô 0/1 chọn mỗi đơn không quá một lần để tổng lợi ích lớn nhất, với tổng τ không vượt B. Dữ liệu của bài là orders và costs vừa tính từ bản đồ; mục tiêu không phải giao tất cả đơn hoặc chọn đơn có lợi ích riêng lớn nhất.
- Đặt F(i,b) là lợi ích tốt nhất khi chỉ xét i đơn đầu tiên và có ngân sách b. Hàng F(0,b)=0. Nếu $\tau_i > b$ thì $F(i,b)=F(i-1,b)$; nếu $\tau_i \le b$ thì $F(i,b)=\max(F(i-1,b), \text{lợi_ích}_i+F(i-1,b-\tau_i))$.
- Hai nhánh là không lấy và lấy đơn i. Nhánh lấy phải dùng hàng $i-1$, bảo đảm mỗi đơn chỉ được chọn một lần. Mỗi ô lưu lời giải tối ưu của một bài toán con; từ các ô trước suy ra ô tiếp theo.
- Trong Julia, table[i+1,b+1] biểu diễn F(i,b), vì chỉ số mảng bắt đầu từ 1. Khi viết nhánh lấy, ô hàng trước tương ứng là $\text{table}[i,b-\tau_i+1]$. Hàng và cột dự trữ cho i=0, b=0 giúp công thức rõ ràng.
- knapsack(orders, costs, budget) trả named tuple gồm ids, value, cost, table. ids là các mã chọn, không lặp; value là tổng lợi ích, cost là tổng τ và table là bảng DP. Trong mã bài, xét danh mục theo ID tăng và đọc τ bằng costs[o.id]. Truy vết từ i=n,b=B để xác định các đơn được chọn; không chỉ trả số tối ưu.
- So sánh với bộ vét cạn đã cung cấp: ba đơn có $2^3=8$ tập con. Giá trị tối ưu phải khớp; nếu có nhiều nghiệm, chấp nhận tập khác nhau khi cùng giá trị tối ưu và đều hợp lệ.
- Thứ tự ưu tiên và chọn tối ưu là hai việc khác nhau. Sắp theo lợi ích/τ giúp tạo thứ tự giao và phương án tham lam để so sánh; tham lam không bảo đảm tối ưu cho ba lô 0/1. Sau DP, lọc các ID chọn theo thứ tự ưu tiên để nạp hàng đợi.

Ví dụ và suy luận

- Ba đơn có (τ,lợi ích) là (3,6),(5,8),(7,10). Với B=8, tập {1,2} có thời gian 8 và lợi ích 14; tập {3} có thời gian 7 và lợi ích 10. Các tập {1,3},{2,3},{1,2,3} đều vượt ngân sách.
- Một ô quan trọng: $F(2,8)=\max(F(1,8),8+F(1,3))=\max(6,8+6)=14$. Với đơn 3, $F(3,8)=\max(F(2,8),10+F(2,1))=\max(14,10)=14$.
- Truy vết từ F(3,8): không lấy đơn 3; lấy đơn 2 rồi giảm ngân sách còn 3; lấy đơn 1 rồi còn 0. Kết quả chọn ID 1,2, tổng thời gian 8, lợi ích 14.
- Quan sát hoạt hình bảng DP theo từng hàng và cột. Trước khi xem giá trị ô mới, sinh viên tính riêng hai phương án lấy/không lấy và giải thích chọn phương án nào.

Lỗi cần tránh

- Truy cập nhầm table[i,b] thay cho table[i+1,b+1], làm lệch cả hàng và cột.
- Nhánh lấy đọc lại hàng i thay vì hàng $i-1$ trong ký hiệu toán, cho phép dùng một đơn nhiều lần.

- Dùng costs[i] khi i chỉ là vị trí của một bản ghi đã đổi thứ tự; từ điển costs phải tra bằng orders[i].id hoặc ID của bản ghi đang xét.
- Chỉ trả F(n,B) mà không có tập chọn hoặc truy vết tập có tổng vượt B.
- Dùng một kết quả tham lam đúng trong ví dụ mini để kết luận tham lam luôn tối ưu.
- Yêu cầu danh sách ID giống hệt một lời giải mẫu dù có nhiều tập tối ưu.

Chi phí và điều kiện áp dụng

- Với n đơn và ngân sách nguyên B, bảng đầy đủ cần O(nB) thời gian và O(nB) bộ nhớ; truy vết cần O(n). Đây là độ phức tạp theo giá trị B, không theo số bit biểu diễn B.
- Vét cạn có 2^n tập con; nếu mỗi tập tính lại tổng bằng duyệt n đơn thì cần $O(n \times 2^n)$. Chỉ dùng làm đối chiếu trên bộ nhỏ.
- Bộ cá nhân có 8–10 đơn nên tối đa 1024 tập con; không suy ra vét cạn phù hợp khi số đơn tăng lớn.

NV4 · Hoàn thiện knapsack(orders, costs, budget)

- Đầu vào: cùng danh mục đơn, từ điển costs ánh xạ ID→τ từ BFS và ngân sách nguyên không âm. Đầu ra: named tuple ids,value,cost,table đúng hợp đồng.
- Khởi tạo hàng/cột cơ sở; điền bảng theo công thức lấy/không lấy trên hàng trước; truy vết ID chọn.
- Kiểm tra ID không lặp, tổng chi phí không vượt budget và tổng lợi ích của tập chọn bằng giá trị DP.
- Đối chiếu với vét cạn đã cho; viết một truy vết trên bộ mini và một kết luận về giới hạn tham lam.

Gợi ý triển khai

- Tạo bảng (n+1)×(B+1) và ánh xạ F(i,b) vào table[i+1,b+1].
- Điền từng hàng theo đơn đang xét. Nếu chi phí lớn hơn b, chỉ có phương án không lấy.
- Truy vết từ hàng cuối theo danh mục ID tăng đã xét: nếu chọn đơn ở hàng i thì ghi ID bản ghi, trừ đúng costs[ID] và chuyển sang i−1; mỗi lần phải giảm i.
- Khi hai phương án bằng nhau, dùng một quy tắc nhất quán; kiểm thử tập hợp và giá trị tối ưu, không áp đặt một tập ID duy nhất.

```
function knapsack(orders,costs,budget)
    # Sinh viên cài đặt tại vị trí TODO trong file chung.
end
```

Lời gọi kiểm tra trong cùng chương trình

```
# Tiền điều kiện: đã chạy bfs_call và hoàn thiện TODO 4.
result = knapsack(data.orders,costs,p.budget)
@assert Set(result.ids) == Set([1,2])
@assert result.value == 14 && result.cost == 8
```

```
julia TH_2026_10_09_Kiem_thu.jl --phase=dp
```

- Chạy julia TH_2026_10_09_Kiem_thu.jl --phase=dp.
- Bộ mini B=8 phải cho tập ID {1,2}, cost=8, value=14; table[end,end]=14.
- B=0 hoặc B nhỏ hơn mọi τ cho tập rỗng, cost=0 và value=0.
- Tổng chi phí/lợi ích tính trực tiếp từ IDs chọn phải khớp kết quả trả về; ID không lặp.
- Giá trị tối ưu của DP bằng vét cạn trên dữ liệu kiểm thử; giữ nguyên dữ liệu khi so sánh.

6. Tích hợp: Một trạng thái giao hàng, một lượt chạy, một báo cáo

Lý thuyết sử dụng

- Kế hoạch chính thức là tập đơn do DP chọn. Danh sách liên kết chứa thứ tự kế hoạch; FIFO chứa các ID đang chờ giao; ngăn xếp chứa lịch sử các lần giao còn hiệu lực. Một bản ghi Order vẫn ở danh mục ngay cả khi không được chọn hoặc đã giao.
- Giao lấy ID ở đầu FIFO, tra cứu BST, dùng tuyến đường và τ đã tính, tăng tổng thời gian và lợi ích, rồi lưu lịch sử. Hoàn tác lấy lần giao trên cùng ngăn xếp, đưa ID trở lại đầu FIFO và trừ đúng hai tổng. Các cập nhật phải cùng thành công hoặc giữ nguyên trạng thái khi có lỗi.
- Bất biến: các ID chờ và đã giao không giao nhau, hợp của chúng đúng bằng tập chọn; tổng đã dùng bằng tổng τ các đơn đã giao; lợi ích đã nhận bằng tổng lợi ích các đơn đã giao; thời gian đã dùng$\le$thời gian kế hoạch$\le B$.
- Danh sách, FIFO, ngăn xếp và BST đã được cung cấp; sinh viên dùng hoạt hình trạng thái và một chuỗi lệnh để giải thích cách chúng tham gia ứng dụng. Không nộp thêm bảy chương trình rời.
- ket_qua.csv là ảnh chụp trạng thái hiện tại. Tệp xuất chứa mọi đơn đầu vào cùng thông tin đường đi, chi phí, dấu được chọn và dấu đã giao theo tiêu đề do mã cung cấp. Các tổng từ dòng đã giao phải khớp tổng ứng dụng.
- Sau bộ mini, có thể chạy dữ liệu theo MSSV: julia TH_2026_10_09_Sinh_vien.jl N24DCDT040 --data rồi julia TH_2026_10_09_Sinh_vien.jl N24DCDT040 --demo hoặc --menu. Thay MSSV ví dụ bằng mã của mình; không chuyển MSSV thành số để tránh mất số 0 đầu.
- Sau buổi học, sinh viên bắt buộc hoàn thiện notebook Julia/Pluto mẫu để trình bày cùng lõi Julia: hiển thị dữ liệu, đường đi, bảng DP và chuỗi giao/hoàn tác, bổ sung ý tưởng và kết quả thực tế. Có thể chạy tệp notebook .jl trực tiếp bằng Julia nếu máy chưa cài Pluto. Mọi mã nguồn, mã sinh CSV, kiểm thử và notebook đều lưu trên GitHub; báo cáo ghi URL và commit cụ thể.

Ví dụ và suy luận

- Sau lập kế hoạch mini: tập chọn {1,2}, FIFO [1,2], lịch sử [], đã dùng 0, nhận 0.
- Giao ID 1: FIFO [2], lịch sử [1], đã dùng 3, nhận 6. Hoàn tác ngay: FIFO [1,2], lịch sử [], đã dùng 0, nhận 0. Kế hoạch không đổi.
- Giao lại ID 1 rồi ID 2: FIFO [], lịch sử [1,2], đã dùng 8, nhận 14. Xuất CSV: ID 1 và 2 đã chọn/đã giao; ID 3 vẫn trong danh mục nhưng chưa chọn và chưa giao.
- Chạy julia TH_2026_10_09_Sinh_vien.jl --mini và julia TH_2026_10_09_Kiem_thu.jl --phase=app sau khi đủ bốn hàm; cuối cùng chạy --phase=all. Ghi kết quả thật, không ghi đạt chỉ vì đã mở được menu.

Lỗi cần tránh

- Hoàn tác đưa đơn về cuối FIFO, làm đổi thứ tự trước thao tác.
- Cập nhật tổng nhưng quên cập nhật FIFO hoặc lịch sử, khiến trạng thái không nhất quán.
- Giao thêm khi FIFO rỗng hoặc hoàn tác khi lịch sử rỗng rồi vẫn thay đổi tổng.
- Xuất kết quả bằng dữ liệu gán tay thay vì đọc từ trạng thái hiện tại.
- Chỉ nộp ảnh màn hình mà thiếu mã nguồn, lệnh chạy hoặc commit trên GitHub.

Chi phí và điều kiện áp dụng

- FIFO vòng lấy/thêm đầu hoặc cuối trong phạm vi sức chứa cho phép cần O(1); ngăn xếp thêm/lấy trên cùng cần O(1) theo thao tác cấu trúc.
- Tra cứu đơn trong BST cần O(h). Chi phí một lệnh giao còn phụ thuộc kiểm tra trạng thái và cách hiển thị tuyến đường; nếu quét toàn bộ kế hoạch để kiểm tra bất biến thì cộng thêm O(n).
- Xuất mọi đơn cần ít nhất O(n), cộng số ký tự biểu diễn các đường đi. Không suy ra toàn bộ lệnh giao là O(1) chỉ từ chi phí FIFO.

Chạy ứng dụng sau khi hoàn thiện bốn hàm

```
julia TH_2026_10_09_Sinh_vien.jl --mini
julia TH_2026_10_09_Kiem_thu.jl --phase=app
julia TH_2026_10_09_Kiem_thu.jl --phase=all
```

Phần demo tính kế hoạch, giao một đơn, hoàn tác rồi giao hết kế hoạch. Theo dõi tuyến đường, hàng đợi, ngăn xếp, thời gian và lợi ích sau từng thao tác. Kết quả được xuất từ trạng thái thực tế, không nhập tay để thay đầu ra.

```
# Tiền điều kiện: đã hoàn thiện cả bốn TODO.
s = run_mini()
@assert s.spent == 8 && s.earned == 14
@assert isempty(queue_values(s.pending))
@assert s.completed == [1,2]
@assert check_invariants(s)
```

Nộp bài và trình bày

- Một repository GitHub cá nhân chứa mọi mã nguồn Julia, mã sinh/đọc CSV đã cung cấp, kiểm thử, notebook Julia mẫu đã hoàn thiện và README hướng dẫn. Nộp URL repository cùng commit đầy đủ của phiên bản nộp; repository riêng tư cần cấp quyền truy cập cho giảng viên.
- TH_2026_10_09_Sinh_vien.jl đã hoàn thiện đúng bốn hàm; giữ cùng mô hình và luồng của khung chương trình. Có thể ghi chú ý tưởng và lỗi đã sửa ngay tại hàm.
- Hai CSV bộ MINI001, một CSV kết quả và bằng chứng chạy các pha kiểm thử. Ghi đúng chuỗi thao tác tạo nên trạng thái xuất.
- Sau buổi học, hoàn thiện một báo cáo ngắn theo sáu phần, khuyến nghị 3–5 trang, tập trung ý tưởng, bảng trung gian và kết quả của cùng ứng dụng; không chép toàn bộ mã vào báo cáo. Phần nộp và trình bày bắt buộc có notebook Julia/Pluto mẫu gọi cùng lõi; cho phép chạy notebook .jl trực tiếp bằng Julia nếu máy chưa cài Pluto.
- Trình diễn đọc CSV → tính thời gian → chọn đơn → giao một đơn → hoàn tác → giao lại → xuất kết quả. Giải thích được ít nhất một bước BFS và một quyết định trong DP.
- Phần phát triển sau buổi: sinh dữ liệu theo MSSV, chạy cùng ứng dụng và truy vết một đơn của mình. Phần này không thay thế kiểm thử bộ mini, không phải yêu cầu làm thêm ứng dụng thứ hai.

Mẫu phiếu kèm theo yêu cầu ghi ý tưởng, bảng truy vết và kết quả kiểm thử. Trình bày bằng notebook Julia/Pluto gọi cùng lõi. Đưa toàn bộ code, bộ sinh/đọc CSV, kiểm thử và notebook lên GitHub, ghi URL cùng commit SHA.

Viết báo cáo theo cùng luồng ứng dụng

Độ dài gợi ý 3–5 trang. Mỗi phần dùng dữ liệu và kết quả thực tế, dẫn tới hàm cùng commit trên GitHub.

1. Bài toán và dữ liệu

- Ghi họ tên, MSSV, ngày thực hành, tên bộ dữ liệu và B; nêu mục tiêu chọn đơn và hai CSV đầu vào.
- Điền: URL GitHub …; commit …; phiên bản Julia …; lệnh tạo/đọc dữ liệu …

2. Sơ đồ xử lý và mô hình

- Vẽ một luồng duy nhất từ CSV tới xuất kết quả. Đánh dấu vị trí của bốn hàm đã hoàn thiện.
- Nêu vai trò của Order, danh mục ID tăng, costs, routes, FIFO, lịch sử và BST; ghi một bất biến trạng thái.

3. Ý tưởng và truy vết thuật toán

- Với mỗi hàm, ghi đầu vào/đầu ra, ý tưởng và điều kiện dừng bằng 3–5 câu; dẫn tới hàm tương ứng trên GitHub.
- Theo dõi cùng ID 2 từ CSV → sắp ID/tra cứu → BFS/τ → DP → trạng thái giao và dòng kết quả. Điền một bảng tìm kiếm nhị phân và giải thích ô F(2,8).

4. Kiểm thử và kết quả thực tế

- Lập bảng gồm pha data/sort/search/bfs/dp/app/all, lệnh đã chạy, kết quả mong đợi, kết quả thực tế và đạt/chưa đạt.
- Đính kèm một lượt giao–hoàn tác và CSV kết quả; nêu ít nhất một lỗi gặp, nguyên nhân và cách sửa. Không ghi đạt cho phần chưa chạy.

5. Phân tích và giới hạn

- Ghi độ phức tạp của bốn hàm theo đúng hiện thực, kể cả bộ nhớ bảng DP và routes đã lưu.
- Giải thích điều kiện áp dụng BFS, giới hạn của tham lam, chi phí BST theo h và lý do chỉ dùng vét cạn với dữ liệu nhỏ.

6. Kết luận và cách chạy lại

- Ghi bốn hàm đã hoàn thiện, kiểm thử đã chạy và phần còn thiếu nếu có.
- Liệt kê các bước tải đúng commit, tạo CSV, chạy kiểm thử, chạy ứng dụng và xuất kết quả. Dẫn tới notebook Julia mẫu đã hoàn thiện gọi cùng lõi; ghi cách mở bằng Pluto hoặc chạy trực tiếp tệp .jl bằng Julia.

Tiêu chí đánh giá đề xuất

Tiêu chí	Điểm	Minh chứng
Đặc tả dữ liệu và đọc hiểu luồng ứng dụng	10	Nêu đúng CSV, đầu vào/đầu ra, công thức thời gian và vị trí của bốn hàm trong một sơ đồ.
Sắp xếp trộn	15	Dãy có thứ tự, đủ phần tử, không sửa đầu vào; xử lý rỗng, đơn lẻ và comparator hòa; giải thích chia/trộn.
Tìm kiếm nhị phân	10	Tra đúng Order/nothing, giữ tiền điều kiện ID tăng, đoạn tìm giảm; có bảng truy vết và thử mã không có.
BFS và chi phí từ bản đồ	20	Khoảng cách, parent, đường đi hợp lệ; không đưa đỉnh nhiều lần; costs thật sự truyền vào DP; giải thích 3,5,7.
Quy hoạch động và truy vết	20	Bảng, chỉ số Julia và tập chọn đúng; cost/value khớp; DP bằng vét cạn; giải thích một ô và nhánh hàng trước.
Tích hợp và kiểm thử trạng thái	15	Trọn luồng chạy được; giao–hoàn tác khôi phục đúng; CSV khớp; ghi kết quả kiểm thử thật và bất biến.
Báo cáo, trình bày và GitHub	10	Báo cáo sáu phần ngắn, lý giải bằng dữ liệu; mọi mã ở GitHub với URL/commit và lệnh chạy lại.

Câu hỏi cuối buổi

1. Vì sao không được dùng dãy đã sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID? Nêu tiền điều kiện của hàm tìm kiếm.

2. Với đơn ID 2 tới điểm 4, phục vụ 1 và bản đồ mẫu, giải thích từng thành phần của thời gian thực hiện τ. BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không?

3. Giải thích hai phương án tạo ô F(2,8) và vị trí của ô đó trong table Julia. Vì sao nhánh lấy đơn phải dùng hàng trước?

4. Sau khi giao ID 1 rồi hoàn tác ngay, FIFO, ngăn xếp, tổng thời gian và tổng lợi ích phải thay đổi thế nào? Nêu một bất biến kiểm tra việc không mất hoặc lặp đơn.

5. Cho biết độ phức tạp theo n,V,E,B của sắp xếp trộn, tìm kiếm nhị phân, BFS cốt lõi và DP bảng đầy đủ. Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n)?

Vị trí trong chương trình học

Chương	Kiến thức	Vai trò
1	Đặc tả, tính đúng đắn, độ phức tạp và đệ quy.	Ghi hợp đồng bốn hàm, điều kiện dừng, bất biến; đọc hàm đệ quy đã cho trên tổng lợi ích của cùng kế hoạch.
2	Tìm kiếm và sắp xếp.	Sinh viên hiện thực merge_sort để chuẩn hóa danh mục/tạo ưu tiên và binary_search để tra cùng ID.
3	Liệt kê cấu hình và vét cạn.	Bộ vét cạn đã cung cấp xét các tập con của chính các đơn để kiểm chứng giá trị DP; sinh viên giải thích 8 tập con của bộ mini.
4	Chia để trị, tham lam và quy hoạch động.	Merge sort minh họa chia để trị; DP là kế hoạch chính thức; phương án tham lam đã cho dùng để so sánh, không thay DP.
5	Danh sách liên kết, ngăn xếp và hàng đợi.	Các cấu trúc đã cho giữ kế hoạch, đơn chờ và lịch sử của cùng ứng dụng; sinh viên kiểm tra giao/hoàn tác bằng trạng thái thật.
6	Biểu diễn đồ thị, DFS và BFS.	DFS đã cho kiểm tra khả năng tới điểm giao; BFS do sinh viên viết cung cấp khoảng cách, đường đi và chi phí đầu vào của DP.
7	Cây nhị phân và cây tìm kiếm nhị phân.	BST đã cho lập chỉ mục cùng danh mục, tham gia lệnh giao và đối chiếu tra cứu; sinh viên giải thích chiều cao h và trường hợp cây lệch.

Bốn hàm là phần cài đặt trong lớp. Các cấu trúc đã cung cấp được đọc, truy vết và vận dụng trong cùng ứng dụng.

Nguồn học tập

- Đề cương chi tiết học phần Cấu trúc dữ liệu và giải thuật
  Nguồn nội bộ đã đọc; xác định kiến thức và nhiệm vụ học phần. Không tự gán các chuẩn đầu ra bị thiếu trong bản trích.

- Bài giảng học phần đã hệ thống hóa
  Nguồn nội bộ đã đọc; ánh xạ bảy chương và lựa chọn phần trọng tâm của một buổi thực hành.
- Đề bài GD-1: Điều phối giao đơn bằng Julia
  Nguồn nội bộ đã đọc; giữ cùng bộ mini, bản đồ, công thức thời gian, trạng thái và quy tắc cá nhân hóa.
- Julia Manual: Getting Started
  Nguồn chính thức đã kiểm tra; áp dụng cách chạy tệp .jl từ cửa sổ lệnh và gọi lại cùng lõi bằng include.
- Julia Manual: Functions
  Nguồn chính thức đã kiểm tra; áp dụng hợp đồng hàm, đối số và kết quả trả về của bốn vị trí cần hoàn thiện.
- Julia Base: I/O and Network
  Nguồn chính thức đã kiểm tra; áp dụng open, đọc dòng và ghi tệp cho CSV số nguyên của buổi thực hành.